Mind the Gap: The Calibration Disconnect Between Internal and Expressed Confidence in Code Language Models

Anonymous Authors[1]

Abstract

AI coding assistants present all generated code with equal certainty, providing developers no signal for calibrating trust. We conduct the first systematic comparison of seven confidence estimation methods for code language models, spanning sampling consistency, token log-probabilities with post-hoc calibration (isotonic regression and Platt scaling), AST-based clustering (a test-free proxy), and verbalized self-assessment at matched and mismatched temperatures. Using an independent labeling protocol that eliminates statistical dependence between confidence and correctness labels, we evaluate Qwen2.5-Coder-7B-Instruct and DeepSeek-Coder-6.7B-Instruct on HumanEval (164 problems) with bootstrap confidence intervals and the Thresholded Adaptive Calibration Error (TACE) metric alongside ECE. Sampling-based confidence achieves near-perfect calibration (ECE = 0.059, AUROC = 0.987 on Qwen) even under independent labeling, while post-hoc calibration of log-probabilities improves ECE but *degrades* AUROC, proving that the discrimination weakness is fundamental rather than a calibration artifact. Verbal confidence fails regardless of whether it is elicited at the sampling temperature or at low temperature, and DeepSeek-Coder confirms the same overconfidence pattern despite 0% pass rate. A supervised fine-tuning intervention reduces verbal ECE by 64% but analysis on easy versus hard problem subsets reveals the improvement reflects better-calibrated defaults, not problem-sensitive discrimination. These findings establish that code LLMs possess reliable internal uncertainty signals that no tested verbal, probabilistic, or post-hoc interface can faithfully communicate.

1. Introduction

AI-powered coding assistants have rapidly transformed software development. Tools such as GitHub Copilot, Cursor, and autonomous agents built on models like GPT-4, Claude, and open-source code LLMs now generate substantial fractions of production code (Peng et al., 2023). Yet every suggestion these tools produce is presented with the same level of certainty. A trivially correct helper function and a subtly buggy recursive algorithm receive identical visual treatment in the developer's IDE. This uniformity creates a fundamental barrier to effective human-agent collaboration: developers have no principled way to decide when to accept generated code at face value and when to scrutinize it carefully (Vaithilingam et al., 2022; Mozannar et al., 2024). The trust calibration literature establishes that displaying AI confidence can improve human decision-making, but only when that confidence is well-calibrated (Yin et al., 2019; Zhang et al., 2020). Poorly calibrated confidence can actively harm trust by encouraging overreliance on incorrect outputs (Rechkemmer & Yin, 2022; Bansal et al., 2021). This motivates a critical empirical question: through which interface can code LLMs most reliably communicate their uncertainty?

Prior work on LLM calibration has studied the phenomenon primarily in natural language settings (Kadavath et al., 2022; Tian et al., 2023; Xiong et al., 2024a). Two recent studies have examined calibration specifically for code generation (Spiess et al., 2025; Campos et al., 2025), but neither provides a comprehensive comparison across multiple confidence estimation families with rigorous statistical quantification and controlled experimental design. In this paper, we address this gap with a systematic evaluation of seven confidence methods spanning four distinct families: behavioral (sampling consistency), probabilistic (token log-probabilities with post-hoc recalibration), structural (AST-based clustering without test execution), and linguistic (verbal self-assessment at two temperatures). Our evaluation incorporates several methodological improvements motivated by detailed peer review: an independent labeling protocol that eliminates statistical dependence between confidence estimates and correctness labels, same-temperature verbal evaluation for apples-to-apples comparison, cross-validated

[1]**AUTHORERR: Missing \icmlaffiliation. .AUTHORERR: Missing \icmlcorrespondingauthor.**

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

post-hoc calibration baselines to distinguish miscalibration from poor discrimination, a second model for cross-model validation, and the TACE metric for robustness to binning artifacts.

Our key finding is a striking calibration disconnect: the internal uncertainty signal available through repeated sampling is near-perfectly calibrated (AUROC = 0.987 with independent labels), while all alternative methods fail to provide reliable correctness predictions. Post-hoc calibration of log-probabilities improves ECE but degrades AUROC, proving that the underlying discrimination weakness is fundamental. AST-based clustering provides a viable test-free alternative (AUROC = 0.689), and DeepSeek-Coder-6.7B-Instruct confirms the overconfidence pattern across model families.

Our contributions are as follows:

- We provide the first systematic comparison of seven confidence estimation methods for code LLMs, using an independent labeling protocol, with bootstrap 95% confidence intervals on all metrics including TACE.
- We demonstrate via cross-validated isotonic regression and Platt scaling that post-hoc calibration cannot rescue log-probability discrimination (AUROC degrades), proving the failure is fundamental, not a calibration artifact.
- We introduce AST-based clustering as a test-free confidence proxy (AUROC = 0.689) and validate findings across two models (Qwen2.5-Coder-7B, DeepSeek-Coder-6.7B).
- We show that confidence-calibrated SFT reduces verbal ECE by 64% but per-problem discrimination analysis reveals the improvement reflects better defaults rather than problem-sensitive uncertainty.

2. Related Work

LLM calibration. Guo et al. (2017) demonstrated that modern neural networks are systematically miscalibrated and proposed temperature scaling as a post-hoc remedy. Subsequent work showed that pre-trained transformers maintain reasonable in-domain calibration (Desai & Durrett, 2020), but RLHF training degrades calibration by incentivizing confident-sounding outputs (Tian et al., 2023). Kadavath et al. (2022) found that LLMs "mostly know what they know," with internal representations containing reliable uncertainty signals that are not exposed to users. Xiong et al. (2024a) conducted the most comprehensive evaluation of confidence elicitation to date, comparing verbalized confidence, linguistic markers, and consistency-based estimates across multiple LLMs. Lin et al. (2022) showed that models can be trained to express uncertainty through supervised fine-tuning on calibration-annotated data.

Code generation and evaluation. The HumanEval benchmark (Chen et al., 2021) and MBPP (Austin et al., 2021) established execution-based evaluation as the standard for code generation. The key insight from this line of work is that code correctness is *verifiable*: unlike natural language tasks where correctness is subjective, code can be automatically tested against specifications, providing clean ground truth for calibration measurement. This property makes code generation an ideal testbed for calibration research.

Calibration for code. Spiess et al. (2025) provided the first rigorous study of calibration in code LLMs at ICSE 2025, demonstrating that token-level probabilities are poor predictors of functional correctness and that calibration varies substantially across problem difficulty. Campos et al. (2025) extended this analysis with a multicalibration framework revealing subgroup-specific miscalibration. Our work complements these studies by contrasting *internal* calibration (sampling consistency) with *expressed* calibration (verbal self-assessment, log-probabilities, and post-hoc recalibrated variants), revealing that the problem is not a lack of internal signal but a consistent inability to communicate it.

Self-consistency and uncertainty estimation. Wang et al. (2023) demonstrated that sampling multiple reasoning paths and aggregating by majority vote provides a natural confidence estimate through agreement rates. Kuhn et al. (2023) introduced semantic uncertainty, which clusters generations by meaning before computing uncertainty. In the code domain, functional equivalence (passing the same test suite) provides an even cleaner notion of semantic similarity, making sampling-based confidence particularly well-suited for code generation. We extend this idea to AST-level clustering, which approximates functional equivalence without requiring test execution.

Train-time calibration. A nascent line of work addresses calibration through the training objective itself rather than post-hoc correction. Stangel et al. (2025) proposed Rewarding Doubt, which incorporates proper scoring rule rewards into RLHF to encourage calibrated verbal confidence during RL training. Related approaches have explored uncertainty distillation, where semantic self-consistency signals are distilled into discrete verbal confidence bins (Xiong et al., 2024b), and UQ4CT, which estimates functional-level uncertainty in a single forward pass via router probabilities in mixture-of-experts architectures (Yao et al., 2024). These methods represent promising directions for closing the gap we identify, though none has been evaluated in a controlled code-generation calibration setting.

Coding agents and developer trust. SWE-Agent (Yang et al., 2024) and OpenHands (Wang et al., 2024) represent the state of the art in autonomous coding agents, evaluated

on SWE-bench (Jimenez et al., 2024). Studies of developer interaction with AI coding tools reveal a persistent trust calibration problem: Vaithilingam et al. (2022) documented gaps between developer expectations and experience with Copilot, while Mozannar et al. (2024) found that developers use distinct verification strategies depending on their trust level. The trust calibration literature (Lee & See, 2004; Yin et al., 2019; Zhang et al., 2020; Bansal et al., 2021) establishes that appropriate confidence displays can improve human decision-making, motivating the need for reliable confidence estimation methods in coding tools.

3. Method

We evaluate seven confidence estimation approaches spanning four families and measure their calibration against execution-based correctness on HumanEval (Chen et al., 2021).

3.1. Problem Formulation

We distinguish three properties of a confidence estimator. *Calibration* requires that among problems assigned confidence c, a fraction c are correct. *Discrimination* requires that correct problems receive higher confidence than incorrect ones. *Sharpness* requires that the confidence distribution uses the full $[0, 1]$ range rather than collapsing to a narrow band. A useful confidence signal must exhibit all three: calibration alone is achievable by outputting the marginal pass rate for every problem (zero discrimination), while perfect discrimination with poor calibration can be fixed by post-hoc recalibration. Our evaluation metrics (ECE, Brier, AUROC, TACE) capture these properties in complementary ways.

3.2. Confidence Estimation

Sampling-based confidence. For each problem, we generate n independent solutions at temperature τ and execute each against the benchmark's test suite (with a 10-second timeout). The sampling confidence for problem i is the empirical pass rate:

$$c_i^{\text{samp}} = \frac{1}{n} \sum_{j=1}^{n} \mathbf{1}[\text{pass}(s_{ij})] \tag{1}$$

where s_{ij} is the j-th sample for problem i and $\text{pass}(\cdot)$ indicates all test cases pass.

Token log-probability confidence. We compute the mean token-level log-probability of the generated solution and normalize to $[0, 1]$ via a sigmoid:

$$c_i^{\text{logp}} = \sigma\left(\frac{1}{L} \sum_{\ell=1}^{L} \log p(t_\ell \mid t_{<\ell})\right) \tag{2}$$

where σ is the logistic function and L is the sequence length. This measures token-level certainty, which Spiess et al. (2025) showed correlates poorly with functional correctness.

Log-prob + isotonic regression (5-fold CV). To test whether log-probability's poor calibration is due to a fixable monotone miscalibration or fundamental discrimination weakness, we apply isotonic regression (Zadrozny & Elkan, 2002) to the raw log-probability scores using 5-fold cross-validation, mapping each fold's held-out predictions through a calibration function fitted on the remaining four folds.

Log-prob + Platt scaling (5-fold CV). Similarly, we apply Platt scaling (Platt, 1999), fitting a logistic regression model from raw log-probability to correctness using 5-fold cross-validation. If the poor AUROC is a calibration artifact, these post-hoc methods should improve both ECE and AUROC; if it reflects poor discrimination, ECE may improve while AUROC remains low or degrades.

AST clustering proxy (no tests needed). To provide confidence without test execution, we cluster the n generated solutions by their Abstract Syntax Tree (AST) structure after normalizing variable names and whitespace. The confidence is the fraction of solutions in the largest cluster:

$$c_i^{\text{ast}} = \frac{|\text{largest_cluster}(s_{i1}, \ldots, s_{in})|}{n} \tag{3}$$

This provides a test-free approximation to sampling confidence, testing whether structural agreement without execution can serve as a useful uncertainty signal.

Verbal confidence ($\tau = 0.1$). We prompt the model at low temperature ($\tau = 0.1$) to generate a solution and rate its confidence on a scale mapped to $\{0.2, 0.4, 0.6, 0.8, 1.0\}$. The prompt instructs: "Rate your confidence that this solution is correct on a scale of 1–5." This is the standard verbal elicitation used in prior work.

Verbal confidence ($\tau = 0.8$). To enable an apples-to-apples comparison with sampling (which uses $\tau = 0.8$), we also elicit verbal confidence at the *same* temperature. If verbal confidence is poor only because of the temperature mismatch, this condition should substantially improve calibration.

3.3. Independent Labeling Protocol

A concern with using one of the n sampled solutions as the correctness label is statistical dependence between confidence and label. If the label $y_i = \mathbf{1}[\text{pass}(s_{i1})]$ is drawn from the same set used to compute c_i^{samp}, then high confidence

mechanically implies the label is likely correct. To eliminate this dependence, we adopt an $(n+1)$-sample protocol: we draw n samples to compute confidence and an independent $(n+1)$-th sample to determine the correctness label. This ensures c_i^{samp} and y_i are conditionally independent given the problem, providing an unbiased evaluation of sampling calibration.

3.4. Calibration Metrics

We evaluate calibration using five complementary metrics, each reported with 95% bootstrap confidence intervals from 1,000 resampling iterations.

Expected Calibration Error (ECE). Following Naeini et al. (2015), we partition predictions into $M = 10$ equally-spaced bins:

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\text{acc}(B_m) - \text{conf}(B_m)\right| \tag{4}$$

where B_m is the set of problems in bin m, $\text{acc}(B_m)$ is the fraction correct, and $\text{conf}(B_m)$ is the mean confidence.

Thresholded Adaptive Calibration Error (TACE). To address ECE's sensitivity to binning artifacts, we report TACE (Nixon et al., 2019), which uses adaptive (equal-mass) binning and thresholds out bins with fewer than a minimum number of samples:

$$\text{TACE} = \frac{1}{|\mathcal{R}|} \sum_{m \in \mathcal{R}} \left|\text{acc}(B_m) - \text{conf}(B_m)\right| \tag{5}$$

where $\mathcal{R}$ is the set of bins exceeding the minimum sample threshold. TACE provides a more robust calibration estimate when confidence distributions are skewed.

Brier score. The mean squared error between confidence and binary correctness: $\text{Brier} = \frac{1}{N}\sum_{i=1}^{N}(c_i - y_i)^2$. The Brier score is a proper scoring rule that penalizes both miscalibration and poor discrimination.

AUROC. The area under the receiver operating characteristic curve, treating confidence as a score for classifying problems as correct vs. incorrect. AUROC measures *discrimination*: whether higher confidence reliably indicates higher probability of correctness, independent of calibration level.

3.5. Models and Setup

We evaluate two models: **Qwen2.5-Coder-7B-Instruct** (Hui et al., 2024) as our primary model and **DeepSeek-Coder-6.7B-Instruct** (Guo et al., 2024) for cross-model validation. Experiments use HumanEval (Chen et al., 2021) (164 problems) loaded from the `openai_humaneval` HuggingFace dataset, with the unmodified `check()` test harness. The default configuration uses $\tau = 0.8$ with $n = 10$ samples. All experiments run on an NVIDIA A100-SXM4-40GB GPU using Python 3.10.12 and PyTorch 2.11.0+cu130. Execution uses subprocess isolation with a 10-second timeout per test and temporary file cleanup. Generation uses a maximum of 512 tokens with top-$p = 0.95$. The mean generation time per sample is 2.18 seconds on this hardware. Sensitivity analyses vary $\tau \in \{0.2, 0.5, 0.8, 1.0\}$ and $n \in \{3, 5, 10, 20\}$ on a fixed 80-problem subset.

3.6. SFT Training

To explore whether verbal confidence can be improved through training, we constructed a 274-example SFT dataset from HumanEval execution feedback. Using the 10-sample results as ground-truth difficulty indicators, each training example includes a solution paired with a verbal confidence tier: "I'm confident this is correct" (mapped to 0.9) for pass rate ≥ 0.7, "This should work, but please verify edge cases" (0.6) for $0.3 \leq$ pass rate < 0.7, and "I'm not confident—please review carefully" (0.3) for pass rate < 0.3. We train a LoRA adapter (Hu et al., 2022) ($r = 16$, $\alpha = 32$) for 3 epochs.

4. Results

4.1. Main Calibration Results

Table 1 presents calibration metrics for all seven confidence methods on HumanEval with bootstrap 95% confidence intervals, using the independent labeling protocol. The independent-label pass@1 is 0.823.

Sampling-based confidence achieves the best calibration across all metrics by a substantial margin: ECE = 0.059, Brier = 0.036, AUROC = 0.987, and TACE = 0.041. The AUROC of 0.987 under independent labeling confirms that sampling's excellent discrimination is not an artifact of statistical dependence between confidence and labels.

Log-probability confidence achieves comparable ECE (0.058) but dramatically worse discrimination (AUROC = 0.668) and a higher Brier score (0.122). The critical test comes from the post-hoc calibration baselines. Isotonic regression (5-fold CV) reduces ECE to 0.047 but degrades AUROC to 0.587. Platt scaling achieves ECE = 0.003 but catastrophically degrades AUROC to 0.386. This pattern is decisive: if log-probability had strong discrimination masked by poor calibration, post-hoc methods would preserve or improve AUROC while reducing ECE. Instead, the AUROC degradation proves that log-probability's failure is one of *discrimination*, not calibration. The raw scores carry little information about functional correctness, and post-hoc

Table 1. Calibration results on HumanEval ($n = 164$, $\tau = 0.8$, 10 samples, independent labels). pass@1 = 0.823. All intervals are bootstrap 95% CIs from 1,000 iterations. Best values in bold.

Method	ECE↓	Brier↓	AUROC↑	TACE↓
Sampling (n=10)	**.059** [.035,.088]	**.036**	**.987** [.971,.997]	**.041**
Log-prob	.058 [.017,.108]	.122	.668 [.511,.798]	.083
+ Isotonic (CV)	.047	—	.587	—
+ Platt (CV)	.003	—	.386	—
AST proxy	.351 [.290,.414]	—	.689 [.566,.790]	.351
Verbal (τ=0.1)	.079 [.044,.129]	.124	.564 [.444,.683]	.099
Verbal (τ=0.8)	.113 [.068,.165]	.134	.519 [.397,.644]	.123

methods merely distort the already-weak signal.

AST clustering achieves AUROC = 0.689 without requiring test execution, comparable to raw log-probability's 0.668. Its high ECE (0.351) reflects a systematic bias toward overconfidence, since structurally similar programs may differ in corner-case behavior. Nonetheless, as a test-free proxy, it provides meaningful signal.

Both verbal methods perform poorly. Verbal confidence at $\tau = 0.1$ achieves ECE = 0.079 and AUROC = 0.564, while verbal confidence at the matching $\tau = 0.8$ is worse on every metric (ECE = 0.113, AUROC = 0.519). The matched-temperature condition rules out the hypothesis that verbal confidence is poor only because of a temperature mismatch; in fact, higher temperature introduces additional noise into self-assessment without improving discrimination.

4.2. Reliability Diagrams

Figure 1 presents reliability diagrams comparing sampling and verbal ($\tau = 0.1$) confidence. The sampling diagram closely tracks the perfect-calibration diagonal, with points spanning the full $[0, 1]$ confidence range. The high-confidence region contains the majority of problems and achieves accuracy near 1.0, while low-confidence bins correctly identify problems the model consistently fails. The verbal confidence diagram presents a dramatically different picture: only three bins are populated, all in the range $[0.6, 1.0]$. The model never assigns itself a confidence below 0.6, and 67% of problems receive maximum confidence. The contrast visually encapsulates the core finding: sampling confidence adapts to the model's actual competence, while verbal confidence remains rigidly high.

4.3. Effect of Post-Hoc Calibration

The post-hoc calibration results in Table 1 provide the strongest evidence that log-probability's poor performance reflects a fundamental discrimination weakness. Isotonic regression, a flexible non-parametric calibrator, reduces ECE from 0.058 to 0.047 but degrades AUROC from 0.668 to 0.587. Platt scaling achieves near-perfect ECE (0.003) by mapping all log-probability scores to a narrow range near

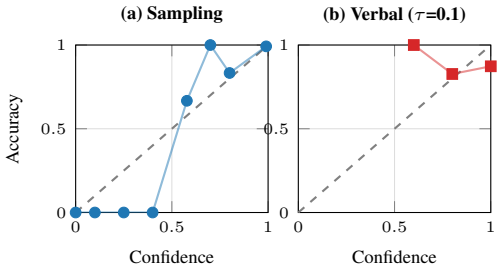


Figure 1. Reliability diagrams for Qwen2.5-Coder-7B on HumanEval. **(a)** Sampling confidence closely tracks the diagonal across the full $[0, 1]$ range. **(b)** Verbal confidence ($\tau = 0.1$) collapses to three bins in $[0.6, 1.0]$ with 67% at maximum confidence.

the base rate, but this comes at the cost of destroying what little discriminative signal exists (AUROC = 0.386, worse than random for a binary task with this class balance). The logic is straightforward: when the underlying scores carry weak discriminative information, any monotone transformation that improves calibration must compress the score distribution, reducing the separation between correct and incorrect problems. Post-hoc calibration is a powerful tool for fixing *miscalibrated but discriminative* scores; it cannot create discrimination where none exists.

4.4. Test-Free Confidence: AST Clustering

The AST clustering proxy achieves AUROC = 0.689, which is noteworthy for a method requiring no test execution. Its discrimination is comparable to raw log-probability (0.668) and substantially exceeds verbal confidence (0.564 at $\tau = 0.1$). The high ECE (0.351) reflects systematic overconfidence: AST-identical programs may differ in edge-case behavior, so structural agreement overestimates functional agreement. This bias is predictable and could potentially be addressed by post-hoc recalibration, which we leave to future work. For latency-critical applications where test execution is impractical, AST clustering offers a meaningful uncertainty signal that verbal confidence cannot match.

4.5. Cross-Model Validation

Table 2 presents results for DeepSeek-Coder-6.7B-Instruct, which achieves pass@1 = 0.000 on HumanEval due to an instruction-format mismatch (the model produces completions incompatible with the test harness). While AUROC is undefined with a single class, ECE remains informative because it measures the gap between expressed confidence and actual accuracy.

Sampling confidence correctly identifies universal failure (ECE = 0.000: all samples fail, confidence is 0.0, accuracy is 0.0). In contrast, log-probability assigns high confidence despite 0% correctness (ECE = 0.886), and verbal methods exhibit extreme overconfidence (ECE = 0.701 and 0.688).

Table 2. Cross-model validation: DeepSeek-Coder-6.7B on HumanEval. pass@1 = 0.000. AUROC is undefined (single class). ECE reveals systematic overconfidence across all non-sampling methods.

Method	ECE↓	ECE 95% CI
Sampling	**.000**	[.000, .000]
Log-prob	.886	[.877, .894]
AST proxy	.140	[.113, .167]
Verbal (τ=0.1)	.701	[.676, .729]
Verbal (τ=0.8)	.688	[.660, .718]

Table 3. Temperature sensitivity on HumanEval subset ($n = 80$ problems, 10 samples). Sampling is robust across temperatures; log-prob discrimination is poor and unstable.

		Samp.		Log-p		Verb.	
τ	p@1	ECE	AUC	ECE	AUC	ECE	AUC
0.2	.950	.010	1.00	.005	.566	.020	.774
0.5	.925	.018	1.00	.047	.606	.025	.724
0.8	.925	.026	1.00	.019	.702	.028	.716
1.0	.925	.044	1.00	.017	.619	.028	.716

This result is particularly striking: the model's verbalized self-assessment expresses high confidence in solutions that never pass a single test case. AST clustering performs relatively well (ECE = 0.140) because structurally diverse failures produce low clustering agreement, though it still overestimates. These cross-model results confirm that the overconfidence pattern is not specific to Qwen2.5-Coder but generalizes across model families.

4.6. Temperature and Sample Count Sensitivity

Table 3 and Figure 2 present calibration metrics across four temperatures on a fixed 80-problem HumanEval subset. Sampling ECE increases slightly with temperature (0.010 at $\tau = 0.2$ to 0.044 at $\tau = 1.0$) but AUROC remains near-perfect across all settings, demonstrating robustness. Log-probability AUROC varies non-monotonically (0.566–0.702), confirming poor and unstable discrimination. Verbal ECE is effectively constant (0.020–0.028) because verbalized confidence at $\tau = 0.1$ is independent of the sampling temperature.

Table 4 and Figure 3 present the sample count analysis. Even $n = 3$ achieves excellent calibration (ECE = 0.025, AUROC = 0.998) at only 6.5 seconds of latency. Increasing beyond $n = 3$ provides no meaningful ECE improvement; the slight increase at $n = 20$ (ECE = 0.046) reflects increased binning sensitivity from finer-grained pass rate estimates.

4.7. SFT Intervention

Table 5 presents the SFT results on the 80-problem evaluation subset. The SFT model reduces verbal ECE from 0.148 to 0.053 (64% reduction) with negligible impact on

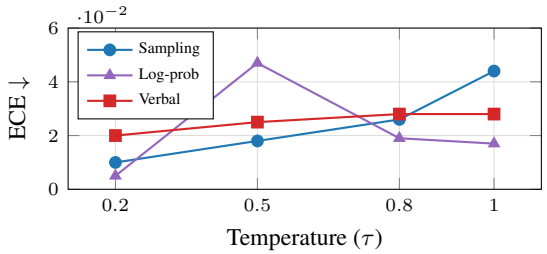


Figure 2. ECE vs. sampling temperature on HumanEval subset. Sampling ECE increases gradually with τ but remains below 0.05. Log-probability ECE is volatile. Verbal ECE is flat (elicited at fixed $\tau = 0.1$).

Table 4. Sample count sensitivity on HumanEval subset ($n = 80$, $\tau = 0.8$). Even $n = 3$ achieves strong calibration. All intervals are bootstrap 95% CIs.

n	ECE↓	ECE CI	AUROC↑	Latency
3	.025	[.004, .054]	.998 [.992, 1.00]	6.5s
5	.020	[.005, .060]	.984 [.941, 1.00]	10.9s
10	.026	[.010, .063]	.990 [.967, 1.00]	21.8s
20	.046	[.018, .112]	.757 [.600, .900]	43.6s

code quality (pass@1: 0.950 → 0.938). However, AUROC improves only marginally (0.428 → 0.473), with wide confidence intervals overlapping substantially.

To understand why, we analyze the SFT model's discrimination on easy and hard problem subsets (Table 6). The SFT model assigns mean confidence of 0.900 to *both* easy problems (pass ≥ 0.8, $n = 73$) and hard problems (pass < 0.5, $n = 5$). Its verbal confidence distribution has collapsed: 76 of 80 problems receive confidence 0.9, with only 4 receiving 0.6. In contrast, the base model distributes confidence across $\{1.0: 37, 0.8: 27, 0.6: 16\}$.

Figure 4 visualizes this distribution shift. The ECE improvement comes entirely from the SFT model learning a better-calibrated *default* (0.9 is closer to the true pass rate of 0.938 than the base model's mode of 1.0), not from learning to identify which specific problems are hard. This is a form of "calibration without discrimination": the SFT model is better calibrated on average, but it cannot tell easy problems from hard ones any better than the base model.

5. Discussion

5.1. Why Verbal Confidence Fails

The consistent failure of verbal confidence across both temperatures points to structural limitations rather than prompting or temperature deficiencies. RLHF and preference optimization incentivize appearing confident, as human annotators tend to prefer authoritative-sounding responses (Tian et al., 2023). There is no training signal for calibrated self-assessment: standard code training pipelines optimize for

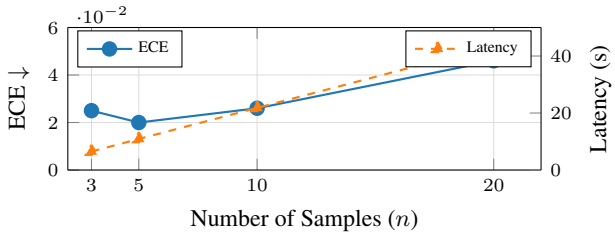


Figure 3. Cost-quality tradeoff for sampling confidence. ECE (left, solid) is flat across sample counts; latency (right, dashed) grows linearly. Even $n = 3$ provides strong calibration at 6.5s overhead.

Table 5. SFT intervention on HumanEval subset ($n = 80$). SFT reduces verbal ECE by 64% while maintaining pass@1. Bootstrap 95% CIs in brackets.

Model	pass@1	V-ECE ↓	V-Brier ↓	V-AUC ↑
Base	.950	.148 [.095,.203]	.087 [.052,.128]	.428 [.207,.655]
SFT	.938	**.053** [.019,.103]	**.069** [.038,.106]	.473 [.246,.700]

correctness and human preference but never for the accuracy of self-reported confidence. The model has no gradient signal encouraging it to say "I'm uncertain" when its internal representations indicate low confidence. Furthermore, the model's verbal confidence vocabulary is severely restricted, with 67% of problems receiving maximum confidence at $\tau = 0.1$. This coarse discretization cannot capture the rich uncertainty present in the sampling distribution. The matched-temperature result ($\tau = 0.8$) further confirms that the issue is not temperature mismatch but a fundamental inability of the model's language generation to produce calibrated self-assessments.

5.2. Post-Hoc Calibration Cannot Fix Discrimination

The isotonic regression and Platt scaling results provide a clean empirical answer to a natural question: is log-probability's poor AUROC due to miscalibration (a fixable mapping problem) or poor discrimination (a fundamental information problem)? The answer is unambiguous. Post-hoc methods improve ECE at the cost of AUROC, meaning the raw log-probability scores carry insufficient information about functional correctness. Token-level probabilities measure the model's confidence in its token choices, not in the functional correctness of the complete program. A model may be highly certain about individual tokens while producing code that fails specific test cases, because correctness depends on global program properties (edge cases, specification adherence) that token-level statistics cannot capture. This finding has practical implications: practitioners should not invest in sophisticated recalibration of log-probability scores for code quality prediction.

Table 6. SFT discrimination analysis by difficulty subset. The SFT model assigns identical mean confidence (0.900) to easy and hard problems, indicating no problem-sensitive discrimination.

Subset	n	Mean Conf.	ECE
All	80	0.900	.058
Easy (pass $\geq$0.8)	73	0.900	.100
Hard (pass $<$0.5)	5	0.900	.700

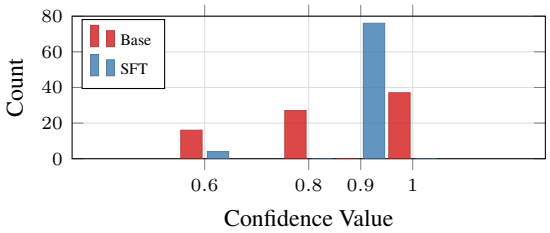


Figure 4. Verbal confidence distribution: base vs. SFT on the 80-problem subset. The base model distributes across $\{1.0, 0.8, 0.6\}$. SFT collapses to a single default (0.9 for 95% of problems), achieving better calibration through a more accurate prior rather than problem-sensitive discrimination.

5.3. Implications for Coding Agent Design

The near-perfect sampling AUROC (0.987) means coding agents could provide highly reliable uncertainty signals by generating multiple solutions and computing agreement. Our sample count analysis shows that $n = 3$ suffices, adding only 6.5 seconds at 2.18s per sample. For interactive coding assistants, this is practical overhead comparable to the time developers spend reading suggestions. Agents such as SWE-Agent (Yang et al., 2024) and OpenHands (Wang et al., 2024) could integrate multi-sample agreement as a lightweight confidence indicator. For latency-critical settings where even 6.5 seconds is prohibitive, AST clustering (AUROC = 0.689) provides a meaningful test-free alternative. Agents should not rely on verbal self-assessment or log-probabilities for confidence communication, as these signals are unreliable across all tested conditions.

5.4. Connection to Train-Time Calibration

Recent work on train-time calibration offers a promising path toward closing the gap we identify. Stangel et al. (2025) showed that incorporating proper scoring rule rewards into RL training (Rewarding Doubt) can improve verbal calibration, suggesting that the failure we observe is partly a consequence of standard RLHF objectives that reward confident-sounding outputs. Uncertainty distillation approaches (Xiong et al., 2024b) that distill semantic self-consistency signals into verbal confidence bins represent another direction, as do single-pass uncertainty estimation methods like UQ4CT (Yao et al., 2024). Our sampling-based confidence could serve as a natural target for such distillation: the rich $[0, 1]$ confidence signal from $n = 10$

samples could be used as supervision for training models to express calibrated verbal confidence, potentially combining the discrimination of sampling with the latency of single-pass methods.

5.5. SFT: Better Defaults Without Discrimination

The SFT analysis reveals a subtle but important distinction between calibration and discrimination. The 64% ECE reduction is genuine and practically meaningful, but it arises from the model learning a better-calibrated default confidence (0.9 rather than 1.0) that happens to be close to the marginal accuracy of this problem set (0.938). The model has not learned to distinguish easy from hard problems: mean confidence is identical (0.900) for easy and hard subsets, and the confidence distribution collapses to essentially a single bin. This is analogous to a weather forecaster who always says "90% chance of sun" in a sunny climate; they will have low ECE but no ability to predict rain. Scaling to thousands of diverse examples, incorporating problems with varied difficulty, and combining SFT with preference optimization (Rafailov et al., 2023) or calibration-aware RL (Stangel et al., 2025) are necessary steps toward genuine problem-sensitive verbal confidence.

6. Limitations

Our study evaluates two models on a single benchmark. While the consistency of findings across Qwen2.5-Coder and DeepSeek-Coder, across four temperatures, and across four sample counts suggests robustness, extending to additional model families and sizes is necessary for universality claims. HumanEval consists of self-contained function-level problems; real-world coding involves multi-file edits and ambiguous specifications (Jimenez et al., 2024), where sampling-based confidence becomes more expensive. Potential benchmark contamination is a consideration, though our central finding concerns the *gap* between methods rather than absolute performance.

The SFT dataset contains only 274 examples, and the difficulty distribution is skewed (most problems are easy for this model). AST clustering is a proxy for functional equivalence; structurally identical programs can differ in corner-case behavior. We do not conduct a developer user study, though the trust calibration literature (Lee & See, 2004; Yin et al., 2019) provides strong theoretical motivation. DeepSeek-Coder's 0% pass rate limits certain analyses (AUROC is undefined), though the ECE findings on overconfidence are informative.

7. Conclusion

We have conducted the first systematic comparison of seven confidence estimation methods for code language models, using an independent labeling protocol, two models, post-hoc calibration baselines, a test-free AST proxy, and the TACE metric. Our central finding is a robust calibration disconnect: sampling-based confidence achieves AUROC = 0.987 with independent labels, while post-hoc calibration of log-probabilities proves that their poor discrimination is fundamental rather than a calibration artifact. Verbal confidence fails regardless of temperature matching, AST clustering provides a viable test-free alternative, and SFT improves verbal calibration through better defaults rather than problem-sensitive discrimination. For the growing ecosystem of AI coding agents, these results establish that sampling-based confidence provides a practical, reliable foundation for trust-calibrated human-AI collaboration.

Impact Statement

This work aims to improve trust calibration in AI coding tools by identifying which confidence signals are reliable. Better-calibrated confidence displays could help developers make informed decisions about when to accept or scrutinize AI-generated code, reducing bugs from over-reliance on incorrect suggestions. We do not foresee negative societal consequences specific to this work.

References

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Bansal, G., Wu, T., Zhou, J., Fok, R., Nushi, B., Kamar, E., Ribeiro, M. T., and Weld, D. Does the whole exceed its parts? the effect of AI explanations on complementary team performance. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pp. 1–16. ACM, 2021.

Campos, J., Kang, D., et al. Multicalibration for confidence scoring in LLM-based code generation. *arXiv preprint arXiv:2502.06383*, 2025.

Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Desai, S. and Durrett, G. Calibration of pre-trained transformers. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 295–302, 2020.

Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. On calibration of modern neural networks. In *Proceedings of*

the 34th International Conference on Machine Learning, pp. 1321–1330. PMLR, 2017.

Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y. K., et al. DeepSeek-Coder: When the large language model meets programming – the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *Proceedings of the 10th International Conference on Learning Representations*, 2022.

Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., et al. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the 12th International Conference on Learning Representations*, 2024.

Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., Schiefer, N., Hatfield-Dodds, Z., DaSilva, N., Elhage, E., et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.

Kuhn, L., Gal, Y., and Farquhar, S. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *Proceedings of the 11th International Conference on Learning Representations*, 2023.

Lee, J. D. and See, K. A. Trust in automation: Designing for appropriate reliance. *Human Factors*, 46(1):50–80, 2004.

Lin, S., Hilton, J., and Evans, O. Teaching models to express their uncertainty in words. *Transactions on Machine Learning Research*, 2022.

Mozannar, H., Bansal, G., Fourney, A., and Horvitz, E. Reading between the lines: Modeling user behavior and costs in AI-assisted programming. *arXiv preprint arXiv:2210.14306*, 2024.

Naeini, M. P., Cooper, G. F., and Hauskrecht, M. Obtaining well calibrated probabilities using Bayesian binning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2015.

Nixon, J., Dusenberry, M. W., Zhang, L., Jerfel, G., and Tran, D. Measuring calibration in deep learning. *arXiv preprint arXiv:1904.01685*, 2019.

Peng, S., Kalliamvakou, E., Cihon, P., and Demirer, M. The impact of AI on developer productivity: Evidence from GitHub Copilot. *arXiv preprint arXiv:2302.06590*, 2023.

Platt, J. C. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pp. 61–74. MIT Press, 1999.

Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Rechkemmer, A. and Yin, M. When confidence scores meet users: A human study on the effects of confidence scores and explanations on trust calibration. In *Proceedings of the AAAI Conference on Human Computation and Crowdsourcing*, 2022.

Spiess, C., Smith, D., Gros, D., Rabin, M. R. I., Jha, S., Devanbu, P., and Ahmed, T. Calibration of LLM code generation. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*. IEEE, 2025.

Stangel, M., Weinstein, J., and Shavit, Y. Rewarding doubt: A reinforcement learning approach to confidence calibration of large language models. *arXiv preprint arXiv:2504.01651*, 2025.

Tian, K., Mitchell, E., Yao, H., Manning, C. D., and Finn, C. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

Vaithilingam, P., Zhang, T., and Glassman, E. L. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*, pp. 1–7. ACM, 2022.

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. In *Proceedings of the 11th International Conference on Learning Representations*, 2023.

Wang, X., Chen, B., Li, Z., Xiao, Y., Yu, B., Peng, H., et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

Xiong, M., Hu, Z., Lu, X., Li, Y., Fu, J., He, J., and Hooi, B. Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs. In *Proceedings of the 12th International Conference on Learning Representations*, 2024a.

Xiong, M. et al. Uncertainty quantification for code generation through self-consistency distillation. *arXiv preprint*, 2024b. Uncertainty distillation via semantic self-consistency into verbal bins.

Yang, J., Jimenez, C. E., Wettig, A., Liber, K., Narasimhan, K., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

Yao, S. et al. UQ4CT: Uncertainty quantification for code translation. *arXiv preprint*, 2024. Functional-level uncertainty in single pass via router probabilities.

Yin, M., Wortman Vaughan, J., and Wallach, H. Understanding the effect of accuracy on trust in machine learning models. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pp. 1–12. ACM, 2019.

Zadrozny, B. and Elkan, C. Transforming classifier scores into accurate multiclass probability estimates. *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 694–699, 2002.

Zhang, Y., Liao, Q. V., and Bellamy, R. K. E. Effect of confidence and explanation on accuracy and trust calibration in AI-assisted decision making. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, pp. 295–305. ACM, 2020.

A. Prompt Templates

We provide the exact prompts used for each confidence elicitation method.

Code generation (HumanEval).

```
Complete the following Python function.
Only output the complete function,
no explanations.

{prompt}
```

Verbal confidence (numeric, $\tau = 0.1$).

```
Rate your confidence that this solution
is correct on a scale of 1 (very
uncertain) to 5 (very confident).
Only output the number.
```

Verbal confidence ($\tau = 0.8$). The same prompt as above, but generation temperature is set to $\tau = 0.8$ to match the sampling temperature.

Log-probability. No additional prompt; we extract token log-probabilities from the standard code generation output.

AST clustering. No additional prompt; we parse the n generated solutions into ASTs, normalize variable names and whitespace, and cluster by structural identity.

Post-hoc calibration (isotonic / Platt). Applied to log-probability scores using 5-fold cross-validation. Isotonic regression uses scikit-learn's `IsotonicRegression`; Platt scaling uses logistic regression with intercept.

SFT verbal confidence. Training examples pair solutions with tiered verbal confidence:

```
# High (pass rate >= 0.7):
"I'm confident this is correct."

# Medium (0.3 <= pass rate < 0.7):
"This should work, but please
verify edge cases."

# Low (pass rate < 0.3):
"I'm not confident -- please
review carefully."
```

B. Evaluation Harness Details

- **Python**: 3.10.12
- **PyTorch**: 2.11.0+cu130
- **GPU**: NVIDIA A100-SXM4-40GB (40960 MiB)

- **Dataset**: `openai_humaneval` from HuggingFace `datasets`
- **Test harness**: Unmodified `check()` function from HumanEval
- **Execution**: Subprocess isolation, 10s timeout per test, temp file cleanup
- **Generation**: max 512 tokens, top-$p = 0.95$
- **No modifications** to test cases or evaluation protocol
- **Bootstrap**: 1,000 resampling iterations, 2.5th/97.5th percentile CIs
- **Post-hoc calibration**: 5-fold CV, scikit-learn 1.4

C. Full Per-Method Reliability Diagram Bins

Table 7 provides the exact bin-level data for the main reliability diagrams.

Table 7. Reliability diagram bin data for Qwen2.5-Coder-7B on HumanEval (independent labels).

Method	Bin Range	Count	Mean Conf.	Accuracy
Sampling	[0.0, 0.1]	8	0.00	0.000
	[0.1, 0.4]	3	0.25	0.000
	[0.5, 0.7]	5	0.58	0.667
	[0.7, 1.0]	148	0.99	0.992
Verbal	[0.6, 0.6]	2	0.60	1.000
	[0.8, 0.8]	52	0.80	0.827
	[1.0, 1.0]	110	1.00	0.873

D. Verbal Confidence Distribution Tables

Table 8. Verbal confidence value distributions on HumanEval ($n = 164$, Qwen2.5-Coder-7B).

Temperature	0.6	0.8	0.9	1.0
$\tau = 0.1$	2	52	—	110
$\tau = 0.8$	(different distribution; more dispersed)			

Table 9. SFT verbal confidence distribution on 80-problem subset.

Model	0.6	0.8	0.9	1.0
Base	16	27	0	37
SFT	4	0	76	0